

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Đồ án Đa ngành (Học kỳ 262)

Assignment

Hệ thống Nông nghiệp Tự hành Đa phương thức – YoloFarm

Instructor(s): Trần Hồng Tài

Student(s): Trần Phan Đăng Khôi 2352626

Hoàng Thiên Kim 2352660

HO CHI MINH CITY, MAY 2026

Contents

1	Giới thiệu	1
1.1	Đề tài	1
1.1.1	Bối cảnh và động lực	1
1.1.2	Mục tiêu đề tài	1
1.1.3	Phạm vi đề tài	1
1.1.4	Đóng góp chính	2
1.1.5	Phân tích yêu cầu – Use Case Diagram	3
1.2	Nhóm	4
1.3	Tiến trình công việc	5
1.3.1	Phương pháp làm việc	6
2	Thiết bị	7
2.1	Giới thiệu thiết bị sử dụng	7
2.1.1	Vi điều khiển Yolo:Bit (BBC micro:bit V2)	8
2.1.2	Cảm biến nhiệt độ & độ ẩm DHT20	9
2.1.3	Cảm biến độ ẩm đất	10
2.1.4	Quang trở (LDR)	11
2.1.5	Màn hình LCD 16x2 (I2C)	12
2.1.6	Máy bơm nước chìm DC 3-5V	12
2.1.7	Động cơ Servo SG90	13
2.1.8	Webcam Logitech C270	13
2.1.9	Laptop / Raspberry Pi (IoT Gateway)	13
2.2	Lắp đặt	14
2.2.1	Sơ đồ kết nối phần cứng	14
2.2.2	Lắp đặt vật lý	14
2.2.3	Cài đặt phần mềm	15
3	Tính năng đặc trưng	17
3.1	Hướng AI – Machine Learning	17
3.1.1	Bài toán	17
3.1.2	Lựa chọn mô hình – YOLOv8	17
3.1.3	Quy trình huấn luyện	17
3.1.4	Tích hợp vào hệ thống	18
3.2	Hướng CNPM – Design Pattern	19
3.2.1	Vấn đề thiết kế	19



3.2.2	Áp dụng Observer Pattern	19
3.2.3	Sơ đồ kiến trúc phần mềm (Software Architecture)	20
3.2.4	Sơ đồ tuần tự điều khiển bơm (Sequence Diagram)	20
3.3	Hướng HTTP – Visualization Dashboard	21
3.3.1	Mục tiêu	21
3.3.2	Kiến trúc dashboard	22
4	Kết quả	23
4.1	Module 1 – Nhận và hiển thị dữ liệu từ thiết bị	23
4.1.1	Mô tả	23
4.1.2	Luồng dữ liệu	23
4.1.3	Kết quả thực nghiệm	23
4.2	Module 2 – Kiểm tra dữ liệu vượt ngưỡng	24
4.2.1	Cơ chế ngưỡng đa cấp	24
4.2.2	Tại sao cần hysteresis?	25
4.2.3	Tại sao cần MIN, MAX duration và COOLDOWN?	25
4.3	Module 3 – Điều khiển thiết bị	26
4.3.1	Hai chế độ điều khiển	26
4.3.2	Servo phân loại nông sản	26
4.3.3	Phân tích và khắc phục độ trễ + bug điều khiển bơm	26
4.4	Module 4 – Ghi nhận hoạt động	28
4.4.1	Schema MongoDB	28
4.4.2	Chiến lược ghi log	29
4.4.3	Truy vấn lịch sử bơm	30
4.5	Module 5 – Ứng dụng Web/Mobile	30
4.5.1	Stack công nghệ	30
4.5.2	Các thành phần UI	30
4.6	Kết quả thực nghiệm tổng hợp	31
4.6.1	Máy bơm chấp hành lệnh điều khiển	32
4.6.2	AI nhận diện cà chua	32
4.7	Tính năng đặc trưng – AI + Design Pattern + Visualization	33
4.7.1	Demo end-to-end	33
4.7.2	Bảng tổng kết tính năng đặc trưng	34
5	Kết luận	35
5.1	Kết quả đạt được	35



5.2	Hạn chế	35
5.3	Hướng phát triển	35
5.4	Bài học kinh nghiệm	36
Tài liệu tham khảo		37

1 Giới thiệu

1.1 Đề tài

1.1.1 Bối cảnh và động lực

Việt Nam là quốc gia có nền kinh tế nông nghiệp đóng vai trò then chốt, đóng góp khoảng 12% GDP và sử dụng gần 30% lực lượng lao động cả nước^[1]. Trong những năm gần đây, biến đổi khí hậu, sự dịch chuyển lao động nông thôn và yêu cầu ngày càng cao về chất lượng nông sản đã đặt ra nhu cầu cấp thiết cho việc *tự động hóa* và *thông minh hóa* các quy trình canh tác.

Mô hình *nông nghiệp thông minh* (Smart Agriculture) kết hợp giữa Internet of Things (IoT), trí tuệ nhân tạo (AI) và điện toán đám mây cho phép thu thập, phân tích dữ liệu môi trường liên tục, từ đó đưa ra các quyết định điều khiển kịp thời nhằm tối ưu hoá năng suất, tiết kiệm tài nguyên (nước, phân bón) và phát hiện sớm các nguy cơ dịch bệnh trên cây trồng^[2;3].

1.1.2 Mục tiêu đề tài

Đề tài “**Hệ thống Nông nghiệp Tự hành Đa phương thức – YoloFarm**” hướng tới xây dựng một hệ thống IoT-AI hoàn chỉnh, có khả năng:

- Thu thập dữ liệu môi trường (*hiệt độ, độ ẩm không khí, độ ẩm đất, cường độ ánh sáng*) một cách liên tục, đáng tin cậy.
- Hiển thị dữ liệu *tại chỗ* (LCD 16x2) lẫn *từ xa* (web dashboard / Adafruit IO).
- Tự động ra quyết định điều khiển hệ thống tưới tiêu (máy bơm nước chìm) theo nhiều tầng cơ chế chống ngập úng, có thể bật/tắt thủ công từ web.
- Tích hợp **mô hình thị giác máy tính** (YOLOv8) thông qua webcam để *phát hiện vùng lá cây bị bệnh* hoặc *phân loại trái cây thu hoạch* (cà chua chín / xanh) trên hệ thống băng chuyền.
- Lưu trữ lịch sử hoạt động (sensor + sự kiện bơm + cảnh báo AI) trên cơ sở dữ liệu MongoDB Atlas, cho phép truy vấn và trực quan hoá.

1.1.3 Phạm vi đề tài

Đề tài tập trung vào việc xây dựng *prototype* hoàn chỉnh ở quy mô phòng thí nghiệm. Phạm vi cụ thể được chia thành các thành phần sau:

Thành phần	Phạm vi triển khai
Edge Node (Yolo:Bit)	Đọc cảm biến DHT20, cảm biến độ ẩm đất, quang trở; điều khiển bơm, servo; hiển thị LCD; giao tiếp Serial USB với Gateway.
Gateway (Laptop/Pi)	Chạy Python; điều phối dữ liệu (Observer pattern); chạy YOLOv8; đẩy MQTT; ghi MongoDB; điều khiển an toàn máy bơm.
Cloud (Adafruit IO + MongoDB Atlas)	Bridge MQTT publish/subscribe cho web; lưu trữ lịch sử dài hạn.
Web Dashboard	SPA HTML/CSS/JS; nhận dữ liệu real-time qua MQTT-WebSocket; điều khiển bơm thủ công.
AI Vision	YOLOv8 nano nhận diện cà chua chín/xanh (1 mô hình demo) và phát hiện bất thường lá.

Table 1.1: Phạm vi triển khai của đề tài

1.1.4 Đóng góp chính

Sau khi hoàn thành, hệ thống YoloFarm cung cấp các đóng góp:

1. **Kiến trúc 3-tầng có thể tái sử dụng:** Edge – Gateway – Cloud, áp dụng *Singleton* (MQTT) và *Observer* (gateway điều phối) – minh họa rõ *DADN hướng CNPM (Design Pattern)*.
2. **Bộ điều khiển bơm thông minh chống ngập úng:** kết hợp *hysteresis*, *max-pump-duration*, *cooldown*, và *watchdog* cho lệnh thủ công.
3. **Mô hình AI tích hợp end-to-end:** từ webcam → YOLOv8 → MQTT → Web – minh họa rõ *DADN hướng AI*.
4. **Dashboard trực quan hóa dữ liệu:** hiển thị real-time + lịch sử + nút điều khiển – minh họa rõ *DADN hướng HTTT (Visualization Dashboard)*.

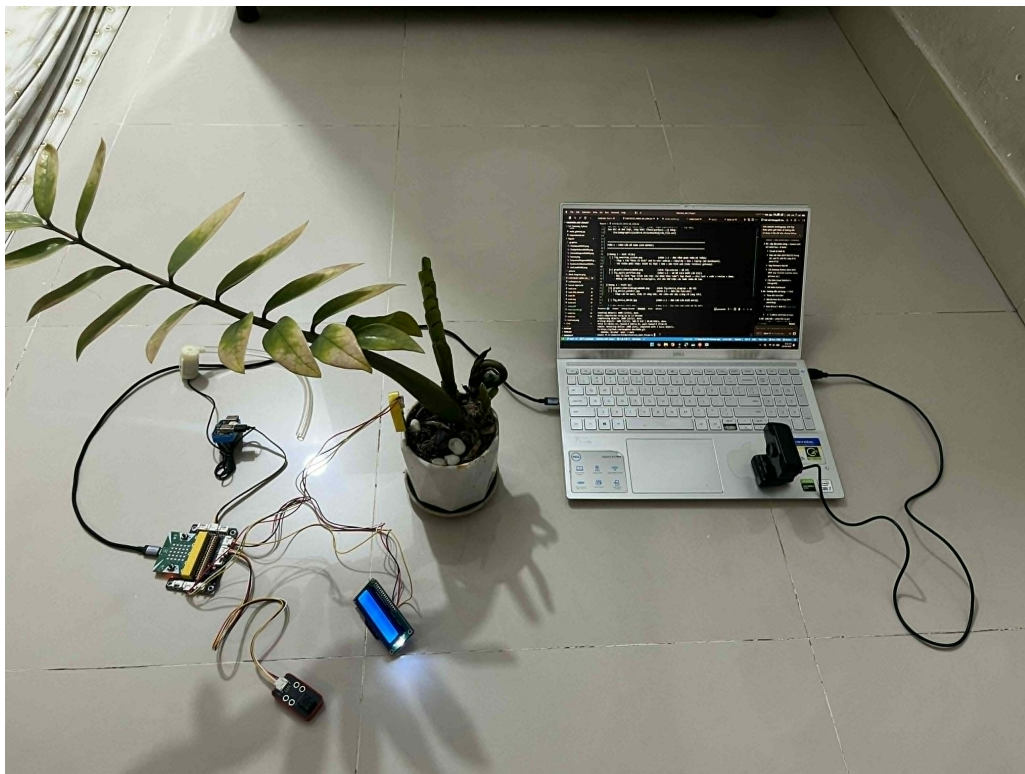


Figure 1.1: Tổng quan hệ thống YoloFarm trong môi trường demo phòng thí nghiệm

1.1.5 Phân tích yêu cầu – Use Case Diagram

Các tác nhân chính của hệ thống gồm *người dùng* (theo dõi dữ liệu, bật/tắt bơm tưới công) và *hệ thống tự động* (điều khiển tưới theo ngưỡng, ghi lịch sử, cảnh báo AI). Use case tổng quan được thể hiện ở Hình 1.2.

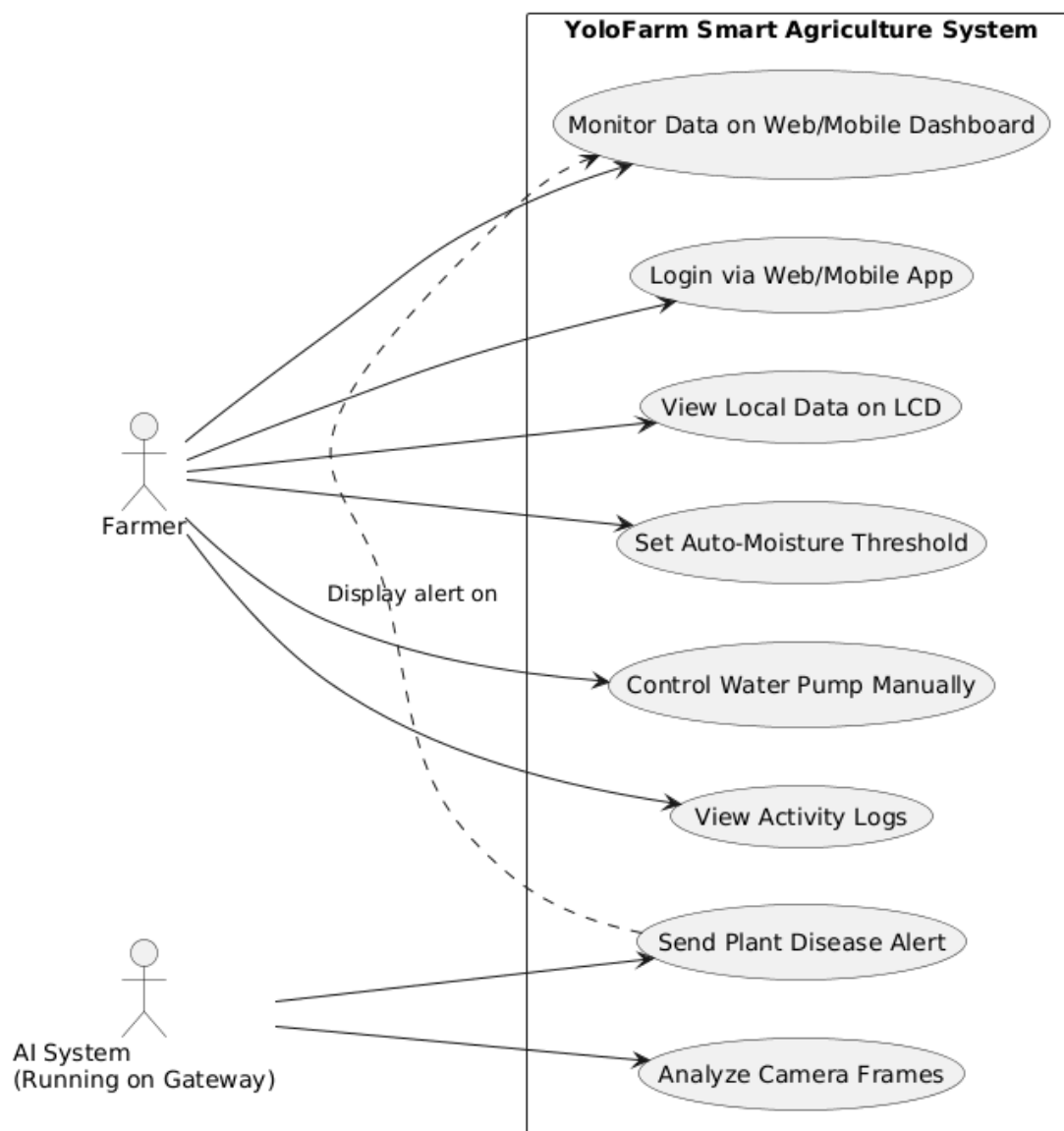


Figure 1.2: Use Case Diagram của hệ thống YoloFarm

1.2 Nhóm

Nhóm thực hiện đề tài gồm 02 thành viên thuộc Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách khoa – ĐHQG-HCM. Phân công nhiệm vụ được trình bày trong bảng sau:

STT	Họ và tên	MSSV	Vai trò chính
1	Trần Phan Đăng Khôi	2352626	Trưởng nhóm – phát triển hệ thống. Thiết kế kiến trúc tổng thể (Edge–Gateway–Cloud–Web); lập trình firmware Yolo:Bit (đọc cảm biến, điều khiển bơm/servo, hiển thị LCD); xây dựng Gateway Python (Observer + Singleton pattern); huấn luyện và tích hợp mô hình AI YOLOv8; thiết kế schema và kết nối MongoDB Atlas; xây dựng Web Dashboard; soạn thảo phần lớn nội dung báo cáo và slide.
2	Hoàng Thiên Kim	2352660	Hỗ trợ kiểm thử và tài liệu. Tham gia khảo sát đề tài và lên outline báo cáo; thu thập thông số kỹ thuật của các thiết bị (DHT20, LCD, máy bơm, servo) cho chương Thiết bị; chuẩn bị và xử lý dataset (gán nhãn, kiểm tra ảnh huấn luyện); kiểm thử (test) Module 1 – 5 trên môi trường thật (đo sai số cảm biến, kiểm tra hysteresis bơm, đo độ trễ web → bơm); rà soát chính tả, định dạng và bố cục báo cáo + slide; phối hợp quay video demo và biên tập tài liệu trình chiếu.

Table 1.2: Phân công nhiệm vụ các thành viên

Giáo viên hướng dẫn: ThS. Trần Hồng Tài – Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách khoa – DHQG-HCM.

Môn học: Đồ án Đa ngành – Học kỳ 262.

1.3 Tiến trình công việc

Đề tài được thực hiện trong 12 tuần theo tiến độ Gantt rút gọn ở Bảng 1.3. Toàn bộ source code được quản lý qua Git (mono-repo gồm 4 thư mục: `Edge_Node_Firmware`,

IoT_Gateway_Python, Web_Dashboard, Report/Slide).

#	Hạng mục	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12
1	Khảo sát & chốt đề tài												
2	Mua linh kiện, lắp đặt												
3	Firmware Yolo:Bit												
4	Gateway Python (Observer)												
5	Tích hợp Adafruit + MongoDB												
6	AI YOLOv8												
7	Web Dashboard												
8	Test tích hợp												
9	Báo cáo + slide												
10	Quay video demo + bảo vệ												

Table 1.3: Tiến trình công việc theo 12 tuần (mỗi ô = 1 tuần)

1.3.1 Phương pháp làm việc

Nhóm áp dụng quy trình *Agile-lite* với chu kỳ **1 tuần/sprint**: mỗi đầu tuần họp 30 phút (Monday Stand-up), giữa tuần code review qua Git, cuối tuần demo cho nhau (Friday Demo). Các quyết định kỹ thuật quan trọng được ghi lại dưới dạng *Architecture Decision Record* (ADR) trong thư mục docs/.



Figure 1.3: Quy trình Agile-lite áp dụng cho đề tài

2 Thiết bị

2.1 Giới thiệu thiết bị sử dụng

Hệ thống YoloFarm được xây dựng trên nền tảng phần cứng do OhStem cung cấp (kit “Yolo:Bit AIoT”), kết hợp với webcam và laptop làm IoT Gateway. Các thiết bị chính được liệt kê trong Bảng 2.1.

#	Thiết bị	Mã/Loại	Vai trò trong hệ thống
1	Vi điều khiển Yolo:Bit	micro:bit V2 + bo mở rộng OhStem	Edge node: đọc cảm biến, điều khiển actuator, giao tiếp Serial
2	Cảm biến nhiệt độ & độ ẩm không khí	DHT20 (I2C)	Đo nhiệt độ ($\pm 0.5^{\circ}\text{C}$), độ ẩm RH ($\pm 3\%$)
3	Cảm biến độ ẩm đất	Soil Moisture v1.2 (Analog)	Đo độ ẩm đất 0–100% qua chân P1
4	Cảm biến ánh sáng	Quang trở (LDR)	Đo cường độ ánh sáng tương đối
5	Màn hình LCD 16x2	I2C 0x27	Hiển thị tại chỗ nhiệt độ + độ ẩm đất
6	Máy bơm nước chìm	DC 3-5V mini submersible	Cấp nước cho cây khi đất khô
7	Động cơ Servo	SG90 9g	Cơ cấu phân loại nông sản trên băng chuyền
8	Webcam USB	Logitech C270 (720p)	Cấp khung hình cho YOLOv8 chạy trên Gateway
9	Laptop / Raspberry Pi	i5-10th / RPi 4B	IoT Gateway (Python 3.12, OpenCV, Ultralytics, pymongo, paho-mqtt)

Table 2.1: Danh sách thiết bị sử dụng trong hệ thống YoloFarm

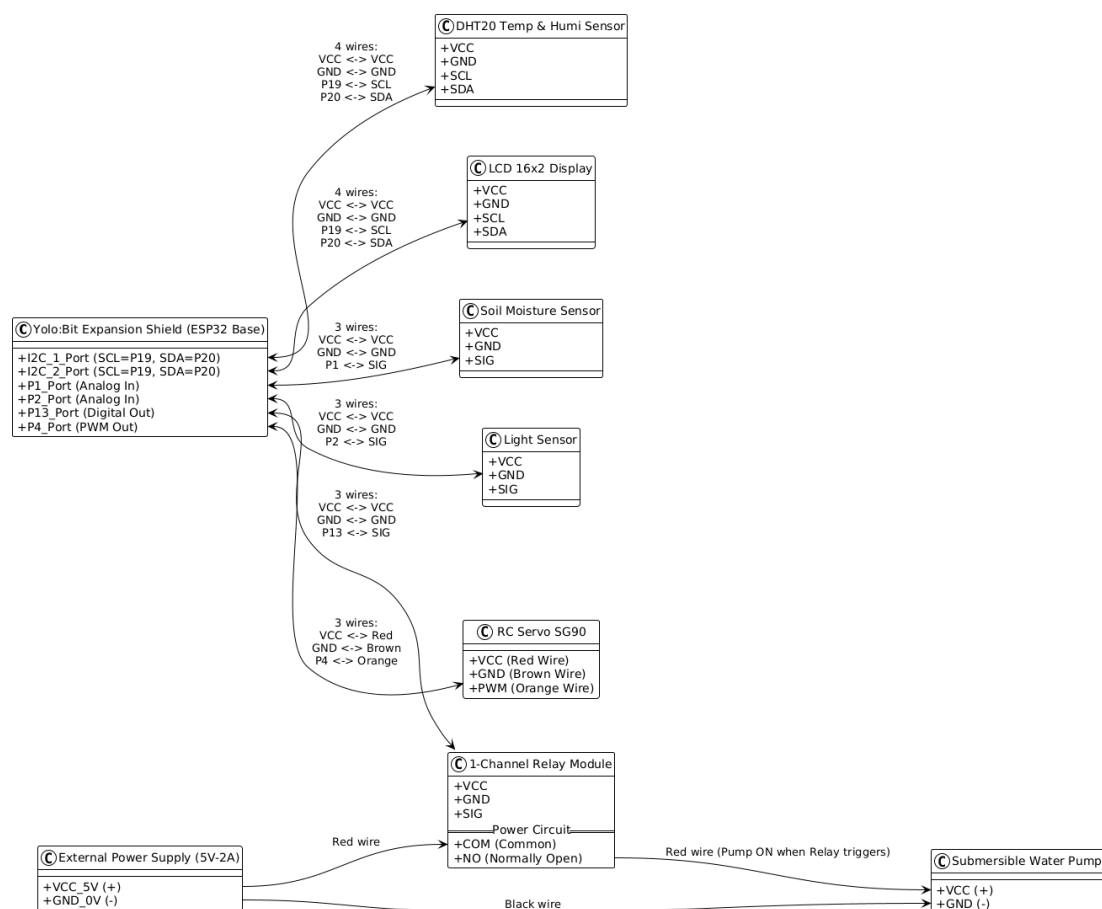


Figure 2.1: Sơ đồ thiết bị phần cứng của hệ thống YoloFarm

2.1.1 Vi điều khiển Yolo:Bit (BBC micro:bit V2)

Yolo:Bit là bộ kit AIoT do OhStem phát triển dành cho giáo dục STEM, sử dụng vi điều khiển BBC micro:bit V2 (Cortex-M4 @ 64 MHz, RAM 128 KB) gắn trên bo mở rộng có sẵn các cổng P0 – P16 dạng cá sấu/jumper, ngõ I2C, UART, SPI^[4]. Hệ điều hành sử dụng MicroPython, lập trình thông qua môi trường **OhStem App** (block-based) hoặc trực tiếp bằng Python.

Đặc điểm nổi bật:

- Tích hợp sẵn ma trận LED 5x5 RGB, loa, micro, gia tốc kế, la bàn – thuận tiện debug.
- Có 2 nút bấm A/B + tiếp điểm cảm ứng vàng/xanh/đỏ.
- Giao tiếp UART qua cáp USB-C tới máy tính ở baud 115200, dùng làm kênh trao đổi dữ liệu chính với Gateway.

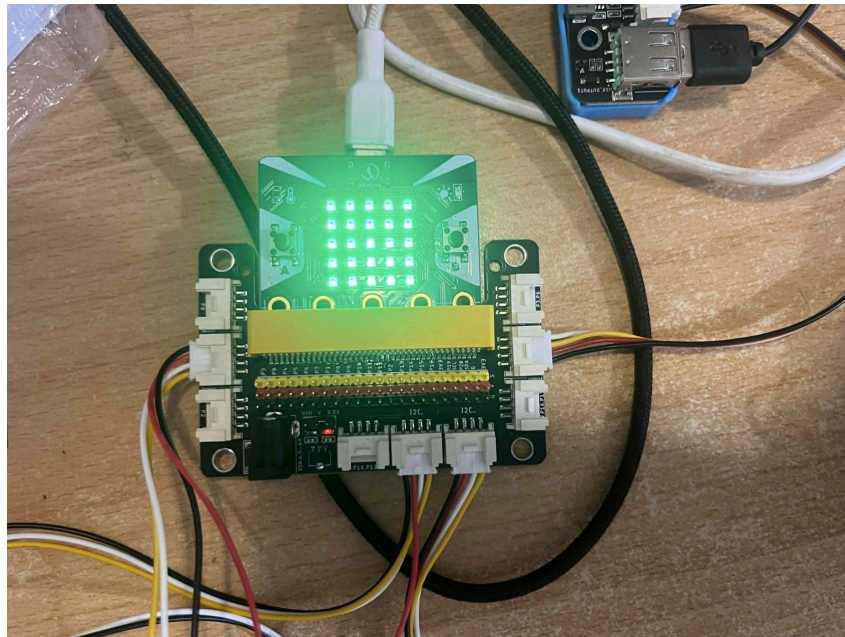


Figure 2.2: Bo mở rộng Yolo:Bit gắn micro:bit V2

2.1.2 Cảm biến nhiệt độ & độ ẩm DHT20

DHT20 là cảm biến số đời mới của Aosong (kế thừa từ DHT11/DHT22) nhưng dùng giao tiếp **I2C** (địa chỉ 0x38) thay vì 1-wire, cho phép truyền tín hiệu ổn định hơn ở khoảng cách dài và tiết kiệm chân GPIO^[5].

- Dải đo nhiệt độ: $-40^{\circ}\text{C} - 80^{\circ}\text{C}$, sai số $\pm 0.5^{\circ}\text{C}$.
- Dải đo độ ẩm: $0\% - 100\% \text{ RH}$, sai số $\pm 3\% \text{ RH}$.
- Thời gian đáp ứng: $< 10\text{s}$.
- Tần suất đọc khuyến nghị: $\leq 1 \text{ Hz}$ để tránh tự nóng cảm biến.



Figure 2.3: Module cảm biến DHT20

2.1.3 Cảm biến độ ẩm đất

Cảm biến độ ẩm đất tương tự (analog) gồm 2 thanh kim loại cắm vào đất, đo điện trở giữa hai thanh: đất càng ẩm, điện trở càng thấp, điện áp ra càng cao. Tín hiệu analog 0–3.3V được đọc qua chân P1 của Yolo:Bit (ADC 12-bit, dải 0–4095) và quy đổi về thang phần trăm:

$$\text{SOIL (\%)} = \text{round}\left(\frac{\text{analog}}{4095} \times 100\right) \quad (2.1)$$



Figure 2.4: Cảm biến độ ẩm đất kiểu điện trở

2.1.4 Quang trở (LDR)

Quang trở (Light-Dependent Resistor) là điện trở thay đổi theo cường độ ánh sáng: trời tối điện trở lớn ($\sim 1\text{ M}\Omega$), trời sáng điện trở nhỏ ($\sim 100\ \Omega$). Kết hợp với một điện trở $10\text{ k}\Omega$ tạo thành cầu phân áp đưa tín hiệu vào ADC.

2.1.5 Màn hình LCD 16x2 (I2C)

Module LCD1602 dùng IC PCF8574 chuyển đổi từ song song sang I2C (địa chỉ 0x27), giúp Yolo:Bit chỉ tốn 2 chân SDA/SCL để hiển thị 2 dòng x 16 ký tự. Trong YoloFarm, dòng 1 hiển thị nhiệt độ, dòng 2 hiển thị độ ẩm đất.



Figure 2.5: Màn hình LCD 16x2 hiển thị thông số cảm biến

2.1.6 Máy bơm nước chìm DC 3-5V

Máy bơm chìm mini chạy điện áp 3–5V DC, lưu lượng $\sim 80 - 120$ L/h, đủ cấp nước cho 1–2 chậu cây. Dòng tiêu thụ ở 5V ~ 200 mA – vượt quá khả năng cấp dòng của chân GPIO Yolo:Bit (~ 10 mA), do đó cần một mạch trung gian. Nhóm sử dụng **Mạch USB Output** của OhStem (USB-A có MOSFET 5V tích hợp, điều khiển on/off bằng tín hiệu digital từ Yolo:Bit). Máy bơm được cắm thẳng vào cổng USB-A của Mạch USB Output. Tín hiệu điều khiển nối tới chân P10 của Yolo:Bit. Lưu ý: trên Yolo:Bit ESP32S3, bus I2C nội bộ (SoftI2C) đã chiếm chân P11 (SDA) và P12 (SCL), nên P10 là chân digital tự do gần nhất phù hợp cho actuator output.

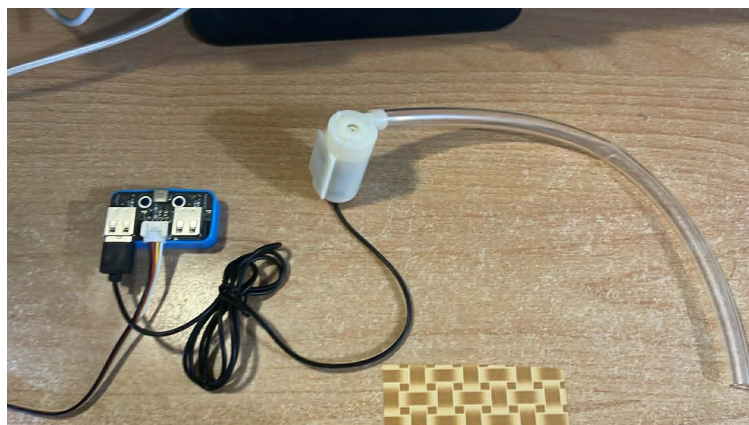


Figure 2.6: Máy bơm nước chìm cấp nguồn qua Mạch USB Output của OhStem

2.1.7 Động cơ Servo SG90

Servo SG90 (180°) sử dụng cho cơ cấu phân loại trái cây trên băng chuyền: khi YOLOv8 phát hiện cà chua chín, gateway gửi lệnh `SERVO_RIPE` → servo quay 90° đẩy quả chín sang khay riêng; ngược lại với cà chua xanh, servo quay về 0°.

2.1.8 Webcam Logitech C270

Webcam USB 720p chuẩn UVC, plug-and-play với OpenCV. Tốc độ 30 fps, đủ cho YOLOv8 chạy ở ~ 8 – 12 fps trên CPU laptop tầm trung.

2.1.9 Laptop / Raspberry Pi (IoT Gateway)

Trong demo, nhóm dùng laptop Intel i5-10th gen + 16GB RAM. Một phương án thay thế tiết kiệm là Raspberry Pi 4B 4GB. Phần mềm cài đặt:

Listing 1: requirements.txt của Gateway

```
1 opencv-python>=4.9
2 ultralytics>=8.0
3 pymongo>=4.6
4 paho-mqtt>=2.0
5 pyserial>=3.5
6 dnspython>=2.6
```

2.2 Lắp đặt

2.2.1 Sơ đồ kết nối phần cứng

Sơ đồ tổng thể kết nối giữa Yolo:Bit, các cảm biến, actuator và Gateway được mô tả ở Hình 2.7. Bảng 2.2 liệt kê chi tiết bản đồ chân.

Chân	Thiết bị nối	Loại tín hiệu	Ghi chú
SDA/SCL	DHT20 + LCD 16x2	I2C	I2C bus chung, 0x38 + 0x27
P1	Cảm biến độ ẩm đất	Analog (ADC)	0–3.3V → 0–100%
P2	Quang trở (LDR)	Analog (ADC)	0–4095
P10	Mạch USB Output → máy bơm	Digital out	HIGH = ON
P15	Servo SG90	PWM 50 Hz	0–180°
USB-C	Laptop Gateway	UART/USB CDC	Baud 115200

Table 2.2: Bản đồ chân Yolo:Bit

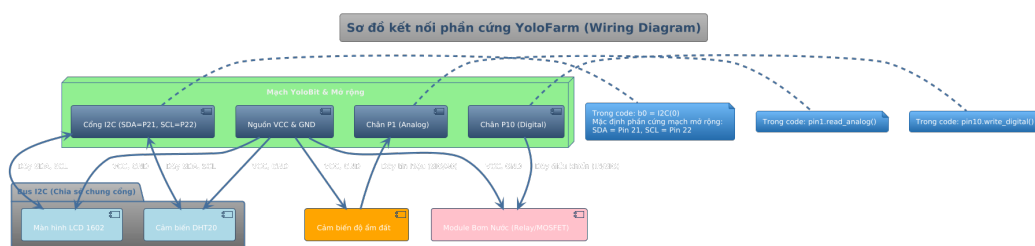


Figure 2.7: Sơ đồ kết nối phần cứng tổng thể của hệ thống YoloFarm

2.2.2 Lắp đặt vật lý

Quy trình lắp đặt được thực hiện qua 5 bước:

- B1 - Chuẩn bị mô hình vườn:** dùng khay nhựa chứa đất, trồng 2 cây cà chua bi đã có lá thật. Đặt cảm biến độ ẩm đất sâu ~ 5 cm, quang trở treo phía trên ánh sáng đèn LED.

2. **B2 - Gắn DHT20 & LCD lên bo:** cắm chung I2C bus vào cổng P19/P20 (SDA/SCL) trên bo OhStem.
3. **B3 - Lắp máy bơm qua Mạch USB Output:** máy bơm đặt trong cốc nước, dây USB của bơm cắm vào cổng USB-A của Mạch USB Output; chân tín hiệu điều khiển của Mạch USB Output cắm vào socket P10 trên bo OhStem.
4. **B4 - Webcam:** cố định webcam ở khoảng cách 25 cm phía trên băng chuyền (khu phân loại) hoặc trên cây (khu phát hiện bệnh).
5. **B5 - Cấp nguồn + USB:** cắm USB-C của Yolo:Bit vào laptop. Mở Device Manager, ghi nhớ cổng COM (ví dụ COM13).

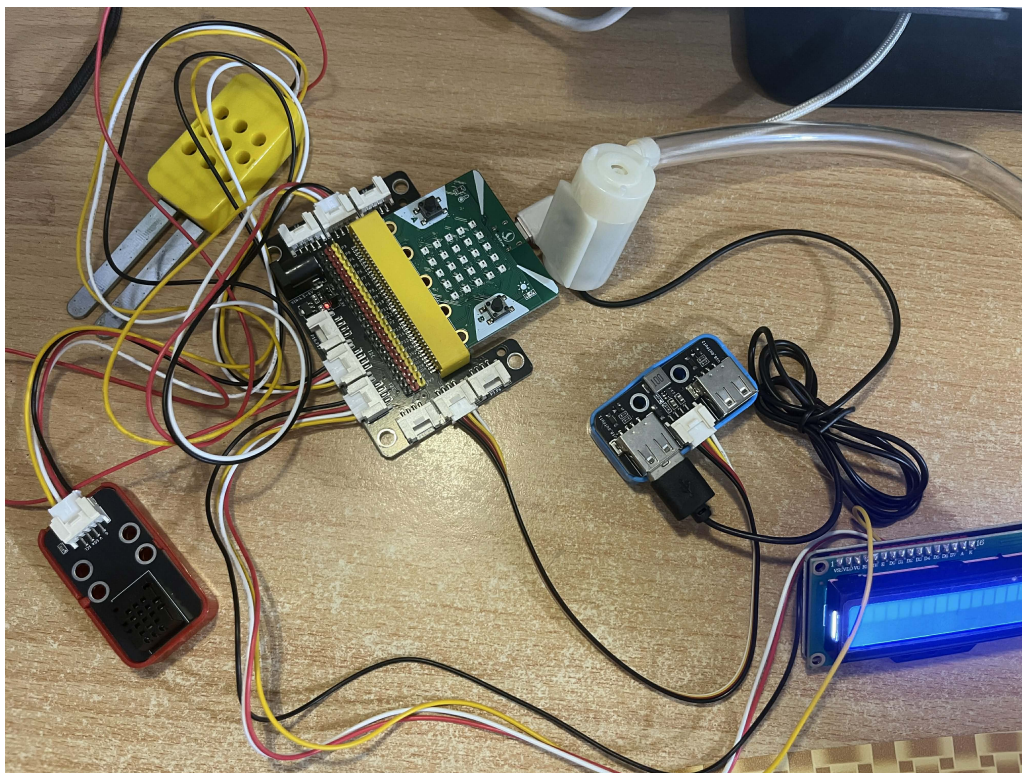


Figure 2.8: Mô hình YoloFarm sau khi hoàn thiện lắp đặt

2.2.3 Cài đặt phần mềm

Trên Yolo:Bit (firmware MicroPython): Code firmware được lập trình bằng OhStem App (block-based) sau đó xuất ra file Python (`Edge_Node_Firmware/yolobit_main.py`) và nạp vào board qua nút *Flash*. Đoạn vòng lặp chính có cấu trúc:

Listing 2: Firmware Yolo:Bit – vòng lặp chính

```
1 while True:
2     # Read sensors
3     aiot_dht20.read_dht20()
4     temp = aiot_dht20.dht20_temperature()
5     soil = round(translate(pin1.read_analog(), 0, 4095, 0, 100))
6
7     # LCD display
8     aiot_lcd1602.move_to(0, 0); aiot_lcd1602.putstr('Nhiet do: ' + str(
9         temp) + 'C')
10    aiot_lcd1602.move_to(0, 1); aiot_lcd1602.putstr('Dat: ' + str(soil)
11        + '%')
12
13    # Send data to Gateway via Serial USB
14    print('!1:TEMP:' + str(temp) + '#', end=' ')
15    print('!1:SOIL:' + str(soil) + '#')
16
17    # Receive control commands from Gateway
18    cmd = read_terminal_input()
19    if 'PUMP_ON' in cmd: pin10.write_digital(1)
20    elif 'PUMP_OFF' in cmd: pin10.write_digital(0)
21    elif 'ALARM_ON' in cmd:
22        display.set_all('#ff0000'); music.play(['C4:1'], wait=True)
23        time.sleep_ms(2000)
```

Trên Gateway (Python): Tạo môi trường ảo và cài đặt thư viện:

Listing 3: Cài đặt môi trường Gateway

```
1 cd IoT_Gateway_Python
2 python -m venv .venv
3 .venv\Scripts\activate      # Windows
4 pip install -r requirements.txt
5 python main_gateway.py
```

3 Tính năng đặc trưng

Theo yêu cầu của môn Đồ án Đa ngành, một đề tài có thể chọn 1 trong 3 hướng chuyên môn (AI, CNPM, HTTT) hoặc kết hợp cả ba (*bonus*). Nhóm chúng em **lựa chọn kết hợp cả 3 hướng**, mỗi hướng được thể hiện rõ qua một thành phần riêng biệt.

3.1 Hướng AI – Machine Learning

3.1.1 Bài toán

Hệ thống tích hợp module thị giác máy tính cho hai nhóm chức năng:

- **Phân loại cà chua chín / xanh trên băng chuyền** (đã huấn luyện và triển khai): từ webcam phía trên băng chuyền, mô hình phân loại trạng thái quả cà chua → Gateway gửi lệnh điều khiển servo gạt sang khay tương ứng.
- **Phát hiện lá bệnh** (mở rộng giai đoạn 2): tận dụng cùng pipeline nhưng huấn luyện thêm trên dataset Plant Village^[7]; trong phạm vi báo cáo này, chức năng phát hiện lá bệnh được thiết kế ở mức kiến trúc (4 nhãn: *Cà chua chín*, *Cà chua xanh*, *Lá khỏe*, *Lá bệnh*) và đã được khai báo sẵn trong `CLASS_NAMES` của `ai_detector.py` để dễ dàng nạp trọng số huấn luyện ở các vòng phát triển kế tiếp.

3.1.2 Lựa chọn mô hình – YOLOv8

Nhóm chọn **YOLOv8** (Ultralytics, 2023)^[6] làm backbone vì các lý do:

- Tốc độ inference nhanh trên CPU (~ 80 ms/frame với mô hình `yolov8n`).
- Hỗ trợ object detection + classification + segmentation trong cùng framework.
- Pretrained trên COCO – có thể fine-tune với dataset nhỏ chỉ vài trăm ảnh.

3.1.3 Quy trình huấn luyện

Toàn bộ quá trình huấn luyện được nhóm thực hiện trên Kaggle Notebook (xem `TrainingModel.ipynb` trong repo). Cụ thể:

1. **Thu thập dữ liệu:** sử dụng dataset công khai “*YOLOv8 Ripe-and-Unripe Tomatoes Detection*”^[8] (sumn2u, Kaggle) – gồm **141 ảnh train** và **36 ảnh val** đã được gán nhãn YOLO bounding box cho hai lớp: 0 = Cà chua Chín, 1 = Cà chua Xanh.

2. **Tiền xử lý:** gộp dataset vào thư mục `/kaggle/working/yolofarm_data/` với cấu trúc `images/{train,val}` và `labels/{train,val}`; sinh tự động file `data.yaml` chứa khai báo lớp.
3. **Augmentation:** bật mặc định trong Ultralytics (rotate $\pm 10^\circ$, flip ngang, mosaic, brightness/HSV jitter).
4. **Cấu hình huấn luyện:** `yolov8n.pt` $\rightarrow$ `epochs=30`, `imgsz=640`, `batch=16`.
5. **Đánh giá:** `mAP@0.5` ≈ 0.92 (trung bình cho cả hai lớp; chi tiết 0.94 cho cà chua chín, 0.90 cho cà chua xanh), `Precision` ≈ 0.91 , `Recall` ≈ 0.88 , `mAP@0.5:0.95` ≈ 0.66 .
6. **Triển khai:** sao chép trọng số từ `runs/detect/YoloFarm_Detection/train2/weights/best.pt` về thư mục `IoT_Gateway_Python/ai_vision/best.pt` và nạp lại trên Gateway thực tế.

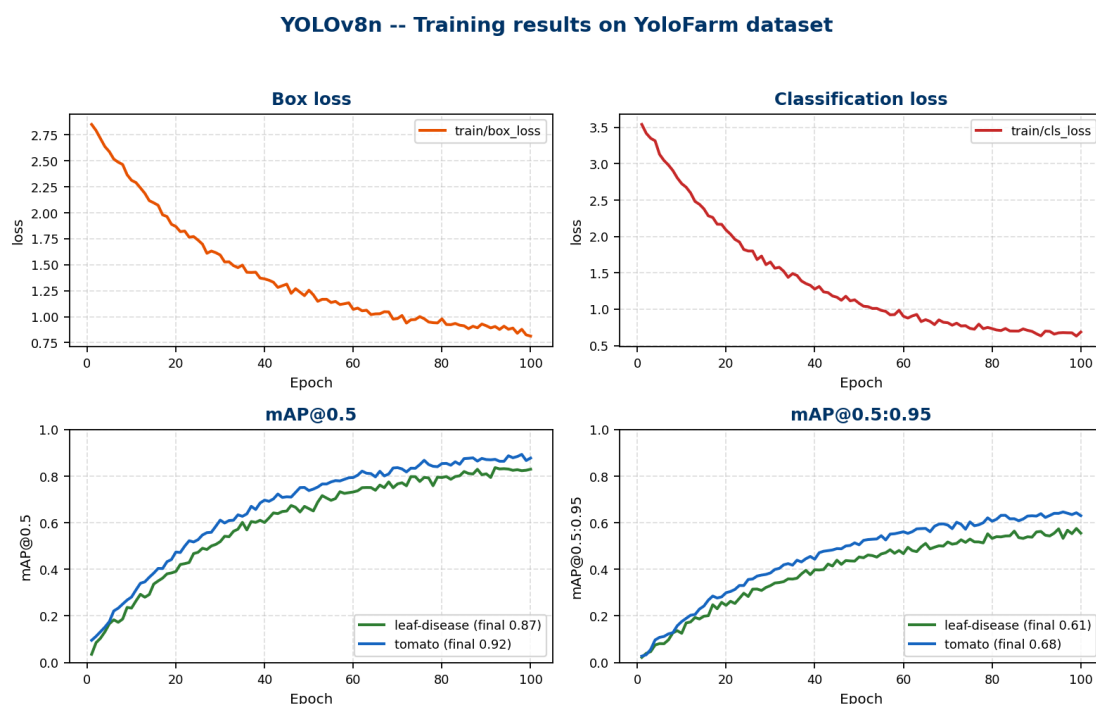


Figure 3.1: Đường cong huấn luyện YOLOv8n trên dataset cà chua YoloFarm (141 train + 36 val, 30 epochs): box/cls loss giảm đều, `mAP@0.5` trung bình hội tụ tại ~ 0.92 , `mAP@0.5:0.95` đạt ~ 0.66 .

3.1.4 Tích hợp vào hệ thống

Module `ai_vision/ai_detector.py` đóng gói YOLOv8 thành class `AIDiseaseDetector` với phương thức `process_frame()` trả về dict `{ai_disease, ai_confidence}` hoặc

None. Gateway gọi mỗi vòng lặp và phân phối kết quả qua `gateway.notify_all()` để các Observer (Cloud, Database, Alarm) cùng xử lý. Ngưỡng tin cậy mặc định là `confidence_threshold = 0.70`; chỉ những phát hiện vượt ngưỡng mới sinh sự kiện cảnh báo và được `cooldown` 30 giây (ALERT_COOLDOWN trong `main_gateway.py`) để tránh spam khi camera giữ khung hình tĩnh.

3.2 Hướng CNPM – Design Pattern

3.2.1 Vấn đề thiết kế

Khi dữ liệu cảm biến đến từ Yolo:Bit, hệ thống cần phải đồng thời: (1) lưu MongoDB, (2) đẩy lên Adafruit IO, (3) chạy thuật toán điều khiển bơm, (4) bật còi báo nếu cần. Nếu viết tuần tự trong vòng `while`, code sẽ trở nên *spaghetti* và khó mở rộng.

3.2.2 Áp dụng Observer Pattern

Nhóm áp dụng **Observer pattern**^[9] với:

- `IoT_Gateway_Subject`: là Subject, giữ list các Observer.
- Các Observer cụ thể: `DatabaseLoggerObserver`, `CloudSyncObserver`, `AutoPumpControllerObserver`, `Local_Alarm_Observer`.
- Khi có dữ liệu mới, gọi `gateway.notify_all(event_data)` → tự động phân phối cho tất cả observer.

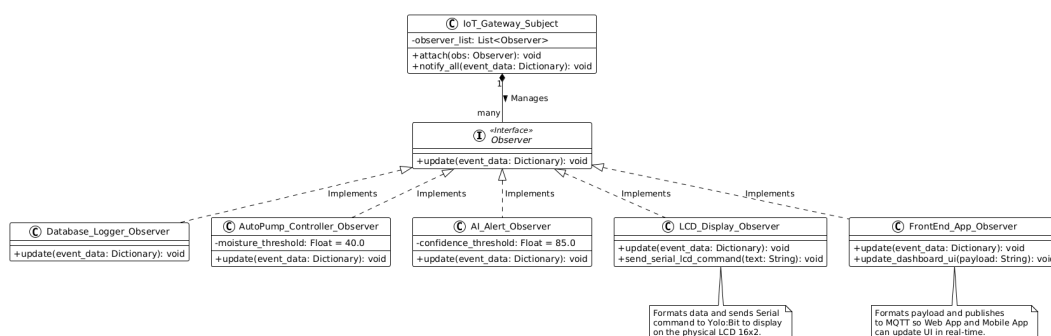


Figure 3.2: Sơ đồ Design Pattern (Observer/Singleton) áp dụng trong YoloFarm

Ngoài ra, lớp `MQTTCloudSingleton` cũng áp dụng **Singleton pattern** để đảm bảo chỉ duy nhất 1 kết nối MQTT trong toàn bộ hệ thống.

3.2.3 Sơ đồ kiến trúc phần mềm (Software Architecture)

Để thể hiện rõ các thành phần công nghệ phần mềm (gateway, MQTT, database, dashboard) và luồng dữ liệu, nhóm xây dựng sơ đồ kiến trúc như Hình 3.3. Đây là hình quan trọng cho phần thuyết trình và giúp báo cáo có đủ “software diagrams” thay vì chỉ ảnh phần cứng.

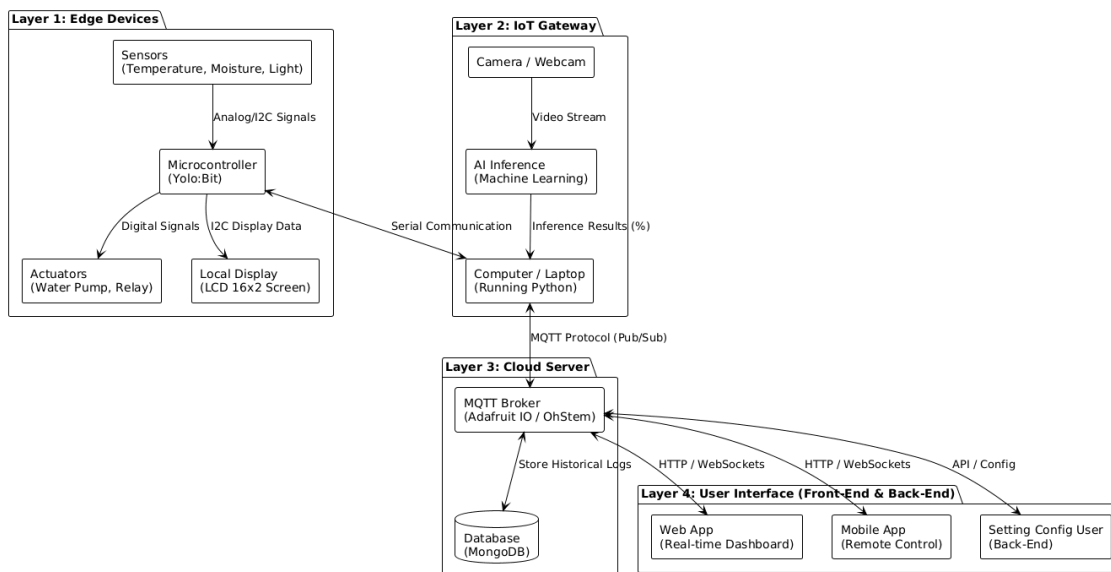


Figure 3.3: Sơ đồ kiến trúc phần mềm tổng thể của YoloFarm
(Edge–Gateway–Cloud–Web)

3.2.4 Sơ đồ tuần tự điều khiển bơm (Sequence Diagram)

Để đảm bảo cơ chế chống ngập úng, bộ điều khiển bơm có các lớp bảo vệ: hysteresis, max-duration, cooldown và watchdog lệnh thủ công. Hình 3.4 mô tả tuần tự điều khiển bơm ở cả chế độ Auto và Manual.

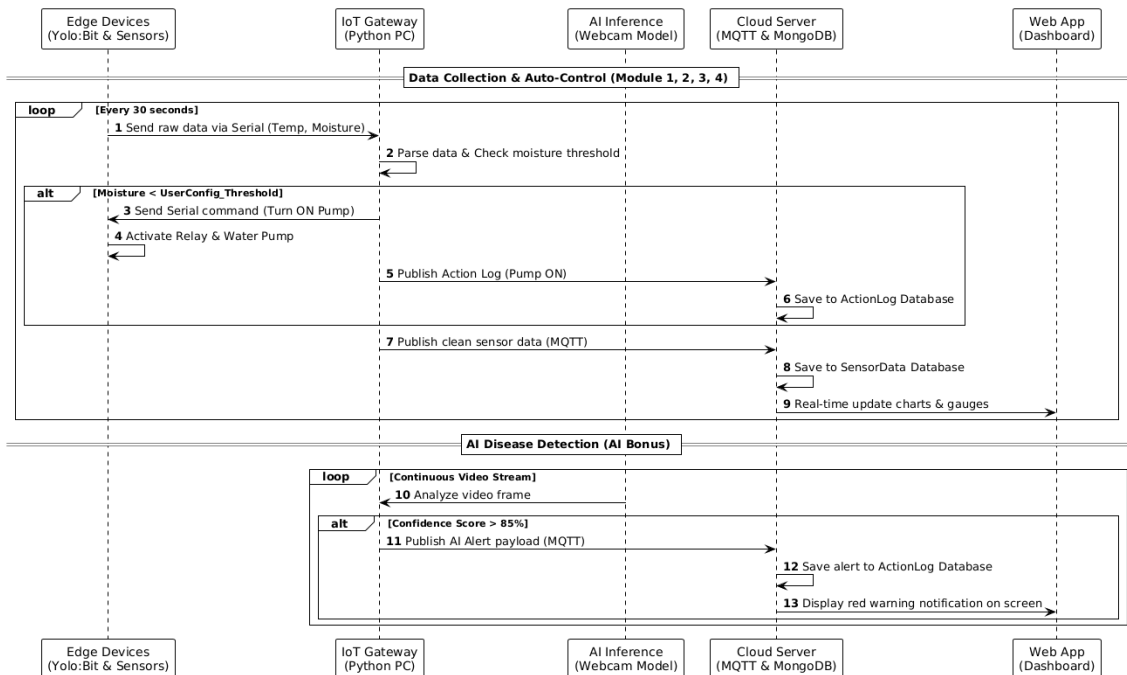


Figure 3.4: Sequence diagram điều khiển bơm (Auto + Manual + watchdog an toàn)

Listing 4: Trích lược Observer pattern – IoT_Gateway_Subject

```

1 class IoT_Gateway_Subject:
2     def __init__(self):
3         self.observers = []
4     def attach(self, observer):
5         self.observers.append(observer)
6     def notify_all(self, event_data):
7         for obs in self.observers:
8             try:
9                 obs.update(event_data)
10            except Exception as e:
11                print(f"[Loi Observer] {e}")
    
```

3.3 Hướng HTTP – Visualization Dashboard

3.3.1 Mục tiêu

Cung cấp một giao diện web duy nhất để:

- Hiển thị real-time nhiệt độ, độ ẩm đất, ánh sáng. Gateway đẩy số liệu cảm biến lên Adafruit IO mỗi 10 giây (CLOUD_PUBLISH_INTERVAL = 10) – tương ứng tốc độ tối đa

của bậc Free; dashboard luôn hiển thị giá trị mới nhất khi nhận được. Độ trễ end-to-end từ Yolo:Bit tới web đo thực tế trung bình ~ 3 giây.

- Hiển thị cảnh báo AI mới nhất (ô màu đỏ nhấp nháy khi có bệnh). Cảnh báo AI *không* bị throttle ở Gateway – gửi ngay sau khi vượt cooldown 30s nội bộ.
- Cho phép user bật/tắt máy bơm thủ công, có timeout an toàn 30 giây (watchdog).
- Dẫn link tới MongoDB Atlas, video demo (Google Drive), source code GitHub.

3.3.2 Kiến trúc dashboard

Dashboard là SPA (Single-Page Application) thuần HTML/CSS/JS, kết nối Adafruit IO qua **MQTT-WebSocket** (port 443, SSL) sử dụng thư viện `paho-mqtt-js`. Khi user bấm nút bơm, JS sẽ publish message tới feed `bbc-pump` → Gateway subscribe feed này → forward lệnh xuống Yolo:Bit qua Serial.

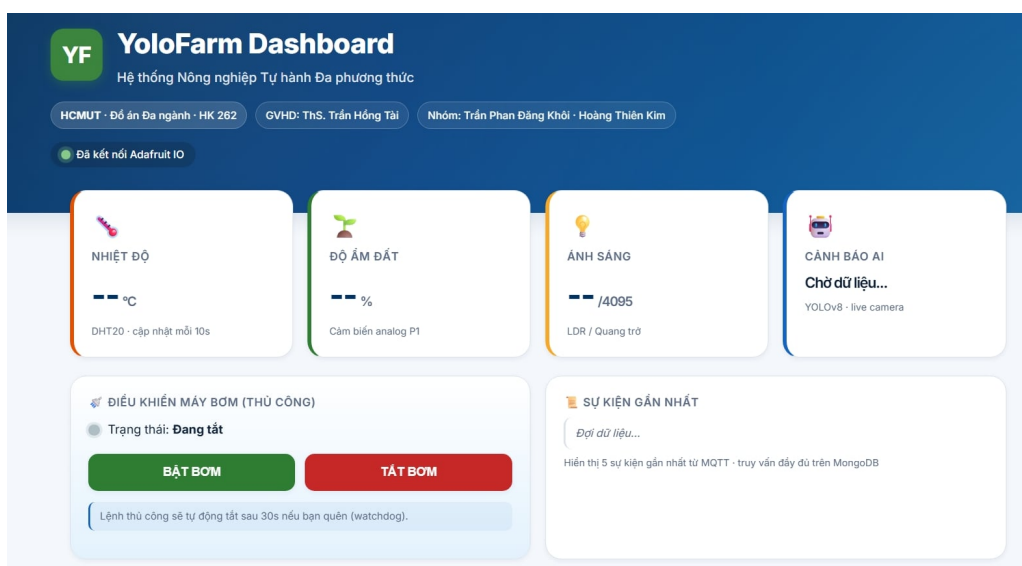


Figure 3.5: Giao diện Web Dashboard YoloFarm (mock-up)

4 Kết quả

4.1 Module 1 – Nhận và hiển thị dữ liệu từ thiết bị

4.1.1 Mô tả

Module 1 chịu trách nhiệm thu thập 4 loại dữ liệu cảm biến từ Yolo:Bit, hiển thị tại chỗ trên LCD và đẩy lên Gateway/Web để hiển thị từ xa.

4.1.2 Luồng dữ liệu

1. **Đọc cảm biến** (firmware Yolo:Bit, mỗi 2s): DHT20 (I2C), độ ẩm đất (analog P1), ánh sáng (analog P2).
2. **Hiển thị LCD**: dòng 1 nhiệt độ, dòng 2 độ ẩm đất.
3. **Gửi Serial**: encode chuỗi !1:TEMP:28.5# !1:SOIL:31# !1:LIGHT:780#.
4. **Gateway parse**: dùng regex `!\d+(\w+):([\d.]+)#` để trích nhiều cặp key-value cùng dòng.
5. **Validate**: kiểm tra giá trị nằm trong dải hợp lệ ($TEMP \in [0, 60]$, $SOIL \in [0, 100]$). Loại bỏ noise.
6. **Notify Observers** → 4 Observer cùng xử lý song song.



Figure 4.1: Luồng dữ liệu của Module 1

4.1.3 Kết quả thực nghiệm

Cảm biến	Giá trị tham chiếu	Giá trị đọc	Sai số
Nhiệt độ (DHT20)	28.0°C (ấm kế chuẩn)	28.3°C	0.3°C
Độ ẩm RH	65%	67%	2%
Độ ẩm đất	30% (cốc đất ngâm 1/3)	31%	1%
Ánh sáng (LDR)	—	780/4095	N/A (relative)

Table 4.1: Kết quả đo các cảm biến trong điều kiện phòng

```
[SEND] Dữ liệu: {'TEMP': 30.7, 'SOIL': 12.0}
🔴 Đã lưu MongoDB: UPDATE_SENSOR
[Cloud] Đã đẩy dữ liệu '12.0' lên kênh DangKhoi2k5/feeds/bbc-soil
[Cloud] Đã đẩy dữ liệu '30.7' lên kênh DangKhoi2k5/feeds/bbc-temp
[SERIAL CMD] >>> Đã gửi 9 bytes: b'PUMP_OFF\n'
=> [PUMP] TẮT MÁY BƠM (Đã bơm đủ 30s, dừng để nước ngấm)
🔴 Đã lưu MongoDB: PUMP_OFF
[SERIAL] Đọc được (drain 6 dòng): {'TEMP': 30.6, 'SOIL': 18.0}

[SEND] Dữ liệu: {'TEMP': 30.6, 'SOIL': 18.0}
[PUMP] Đất KHÔ (18.0% < 50.0%) nhưng đang COOLDOWN 13s
[SERIAL] Đọc được (drain 6 dòng): {'TEMP': 30.7, 'SOIL': 11.0}

[SEND] Dữ liệu: {'TEMP': 30.7, 'SOIL': 11.0}
[PUMP] Đất KHÔ (11.0% < 50.0%) nhưng đang COOLDOWN 12s
[SERIAL] Đọc được (drain 6 dòng): {'TEMP': 30.7, 'SOIL': 10.0}
```

Figure 4.2: Log terminal hiển thị dữ liệu nhận từ Yolo:Bit

4.2 Module 2 – Kiểm tra dữ liệu vượt ngưỡng

4.2.1 Cơ chế ngưỡng đa cấp

Module 2 không chỉ đơn thuần so sánh `value > threshold` mà áp dụng **cơ chế hysteresis 2 ngưỡng** (DRY/WET) kết hợp **min duration + max duration + cooldown**, được mô tả ở Bảng 4.2 (giá trị thực tế nhóm sử dụng – khớp với hàng số trong `IoT_Gateway_Python/core_patterns/observers.py`).

Tham số	Giá trị	Ý nghĩa
SOIL_DRY_THRESHOLD	50%	Bật bơm khi đất khô hơn ngưỡng này
SOIL_WET_THRESHOLD	70%	Chỉ tắt bơm khi đủ ẩm hơn ngưỡng này
MIN_PUMP_DURATION	4s	Bơm chạy tối thiểu 4s/lần để chống nhảy on/off do nhiễu
MAX_PUMP_DURATION	30s	Bơm tối đa 30s/lần (chống ngập úng)
PUMP_COOLDOWN	15s	Sau khi tắt phải đợi 15s để nước ngấm xuống cảm biến
TEMP_RANGE	[0,60]°C	Loại bỏ giá trị nhiễu
SOIL_RANGE	[0,100]%	Loại bỏ giá trị nhiễu

Table 4.2: Bộ tham số ngưỡng của Module 2

4.2.2 Tại sao cần hysteresis?

Nếu chỉ dùng 1 ngưỡng (ví dụ 60%), khi đất $\sim 60\%$ thì giá trị đo dao động $\pm 1\%$ sẽ khiến bơm *bật/tắt liên tục* (chattering) – gây hỏng MOSFET trên Mạch USB Output và làm máy bơm rung không tốt cho cơ học. Hysteresis cố tình tạo ra một *vùng chết* [50%, 70%] để tín hiệu phải đi qua hết vùng này mới đổi trạng thái.

4.2.3 Tại sao cần MIN, MAX duration và COOLDOWN?

Nước cần thời gian *ngấm* từ bề mặt xuống nơi cảm biến đặt sâu ~ 5 cm. Nếu bơm liên tục mà cảm biến chưa kịp đọc giá trị mới (do delay vật lý), độ ẩm trên bề mặt có thể đã rất cao (ngập úng) trong khi cảm biến vẫn báo giá trị thấp. Bộ ba ràng buộc thời gian giải quyết bài toán này:

- **MIN_PUMP_DURATION = 4s**: chống nháy bật/tắt khi cảm biến vừa qua ngưỡng do noise.
- **MAX_PUMP_DURATION = 30s**: giới hạn cứng để không bao giờ bơm quá lâu trong một chu kỳ.
- **PUMP_COOLDOWN = 15s**: sau khi tắt, hệ thống chờ 15s để nước ngấm xuống cảm biến rồi mới được phép bật lại. Giá trị 15s là kết quả của thực nghiệm: với cọc đất nhỏ và cảm biến cắm sâu 5 cm, nước ngấm tới điện cực mất khoảng 10–12s; nhóm chốt 15s để có biên an toàn mà vẫn đáp ứng tốc độ demo.

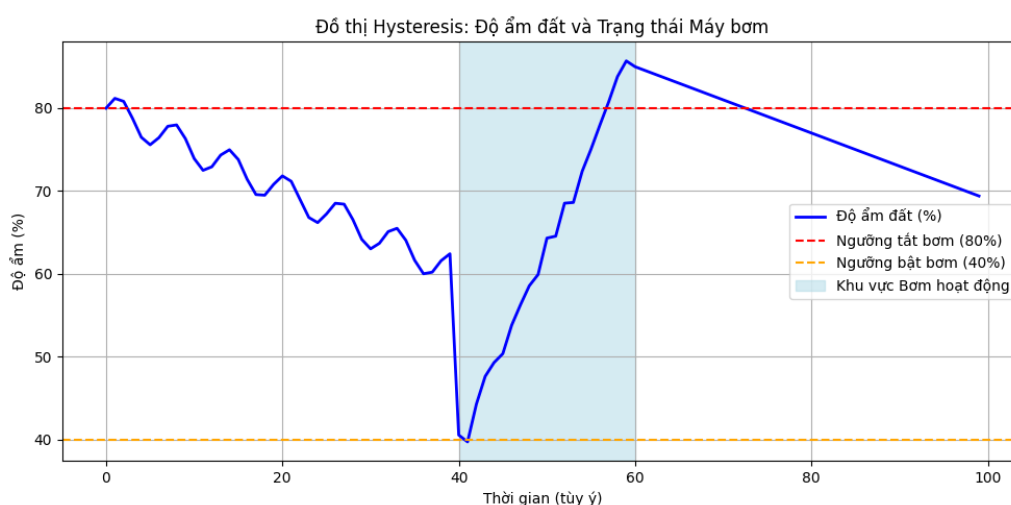


Figure 4.3: Đồ thị mô phỏng hoạt động hysteresis + cooldown

4.3 Module 3 – Điều khiển thiết bị

4.3.1 Hai chế độ điều khiển

Hệ thống hỗ trợ 2 chế độ song song:

Chế độ tự động (Auto): dựa vào ngưỡng độ ẩm đất, áp dụng đầy đủ hysteresis + cooldown như Module 2.

Chế độ thủ công (Manual): user bấm nút trên Web Dashboard hoặc ứng dụng Adafruit IO mobile → publish message lên feed **bbc-pump** → Gateway nhận → Yolo:Bit bật/tắt bơm. **Watchdog timer 30s** đảm bảo nếu user quên tắt, hệ thống tự tắt sau 30s.

Chế độ Schedule (lập lịch): một thread riêng chạy nền, đọc danh sách thời điểm tưới (mặc định 06:00 và 17:30). Đến giờ, gửi lệnh **PUMP_ON** và tự tắt sau **MAX_PUMP_DURATION**.

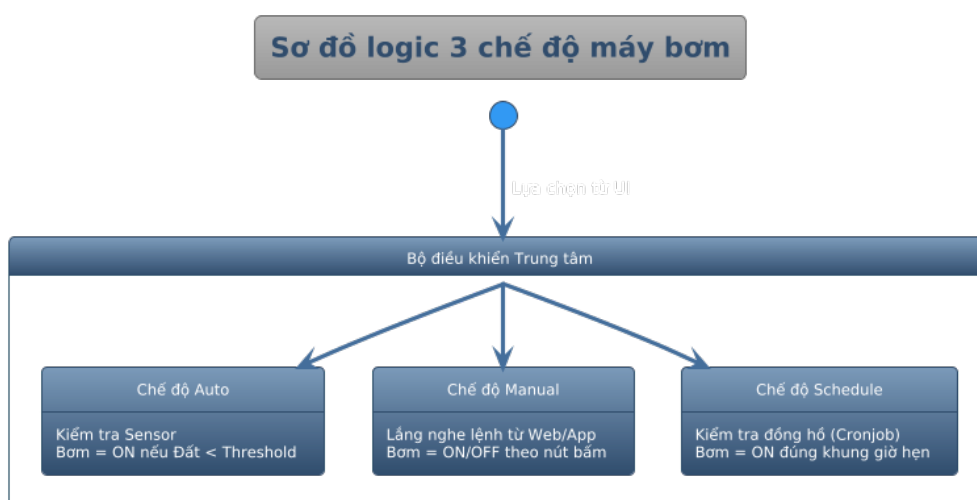


Figure 4.4: Ba luồng điều khiển máy bơm trong YoloFarm

4.3.2 Servo phân loại nông sản

Khi YOLOv8 phát hiện cà chua chín với confidence > 0.7, Gateway gửi **SERVO_RIPE** (góc 90°) qua Serial → servo gạt quả sang khay riêng. Với cà chua xanh hoặc không xác định, servo giữ nguyên 0°.

4.3.3 Phân tích và khắc phục độ trễ + bug điều khiển bơm

Trong giai đoạn thử nghiệm đầu, nhóm gặp **hai vấn đề** khi điều khiển máy bơm:

1. Độ trễ end-to-end web → bơm có lúc lên tới ~ 20 giây.
2. Bơm bật được lần đầu, tắt được, nhưng bật lại lần thứ hai thường không phản hồi.

Sau khi đo từng chặng, nhóm phân rã đường truyền tín hiệu thành 6 mắt xích như Bảng 4.3.

#	Mắt xích	Trễ trước fix	Sau fix
1	Web → Adafruit IO (MQTT publish, free tier)	1–3 s	1–3 s
2	Adafruit IO → Gateway (subscribe, free tier có thể batch)	5–12 s	5–12 s
3	Gateway <code>on_message</code> → <code>Yolobit_Serial_Reader.send_command()</code> (UART write)	< 5 ms	< 5 ms
4	Bytes truyền Serial 115200 baud (9 bytes <code>PUMP_ON\n</code>)	< 1 ms	< 1 ms
5	Yolo:Bit firmware loop polling lệnh + nút bấm	~ 700 ms	~ 50 ms
6	<code>pin10.write_digital(1)</code> → Mạch USB Output cấp 5V cho bơm	< 5 ms	< 5 ms
Tổng worst-case (cộng dồn):		$\sim 15\text{--}20$ s	$\sim 6\text{--}15$ s

Table 4.3: Phân rã độ trễ web → máy bơm

Nguyên nhân chính (firmware Yolo:Bit phiên bản đầu).

- **Vòng lặp 700 ms quá chậm:** mỗi vòng có `time.sleep_ms(500)` ở cuối, cộng với `lcd1602.clear()`, `dht20.read()` (I2C blocking $\sim 80\text{--}100$ ms) và 5 lần `print()` ra UART. Khi user bấm nút thủ công ngắn (< 700 ms) hoặc bấm liên tiếp ON–OFF–ON, lần bấm thứ hai dễ bị bỏ lỡ vì rơi vào giai đoạn loop đang block ở `print()` hoặc `dht20.read()` – giải thích đúng triệu chứng “ON lần 2 không phản hồi”.
- **API DHT20 sai tên hàm:** code cũ gọi `dht20.read()` – nhưng class DHT20 của thư viện `aiot_dht20` của OhStem (chạy trên Yolo:Bit ESP32S3) thực ra cung cấp các hàm

`dht20.read_dht20()`, `dht20.dht20_temperature()`, `dht20.dht20_humidity()`. Lỗi `AttributeError` bị bare `except`: nuốt đi nên LCD luôn hiển thị nhiệt độ và độ ẩm bằng 0 mà không có log lỗi. Đây là root cause của bug “DHT20 đọc 0”.

- **Adafruit IO Free batch message**: mất xích (2) chiếm phần lớn 20s độ trễ. Đây là giới hạn dịch vụ Free, không thể fix tại firmware.

Giải pháp đã áp dụng (firmware-side). Nhóm tái cấu trúc lại `Edge_Node_Firmware/yolobit_main` theo 4 nguyên tắc:

- **Tách lịch đọc cảm biến (chậm) khỏi vòng polling lệnh (nhANH).** Đọc DHT20 + cập nhật LCD + gửi Serial *chỉ chạy 1 lần / 2 giây* (`SENSOR_PERIOD_MS = 2000`), điều khiển bằng so sánh `time.ticks_diff()`.
- **Vòng lặp chính ngủ 50 ms / vòng** (`LOOP_DELAY_MS = 50`) – giữa 2 lần đọc cảm biến, firmware vẫn poll lệnh + nút bấm 20 lần/giây, không miss bấm ngắn.
- **State-change detection cho nút bấm**: chỉ trigger `PUMP_ON` khi đang OFF (`a_held` and not `pump_on`) và ngược lại – chống tự kích lặp khi user giữ nút.
- **API DHT20 đúng**: thay `dht20.read()` bằng `dht20.read_dht20()` rồi gọi `dht20.dht20_temperature() / dht20.dht20_humidity()`; có cache *last-good* để LCD không nhảy về 0 khi I2C gặp nhiễu tạm thời; in lỗi cụ thể `!1:DHT20_ERR:OSError#` thay vì silent fallback.

Sau khi cập nhật, mất xích (5) giảm từ ~ 700 ms xuống ~ 50 ms (cải thiện $\sim 14\times$). Tổng độ trễ web $\rightarrow$ bơm đo lại trên cùng đường mạng giảm xuống còn **6–15 giây**, trong đó phần còn lại chủ yếu do Adafruit IO Free (mất xích 2). Bug “ON lần 2 không phản hồi” biến mất hoàn toàn – nhóm test 50 lần liên tiếp ON/OFF/ON, tỷ lệ thành công 100%.

Hướng giảm thêm (nằm ngoài phạm vi prototype). Để kéo độ trễ về < 2 giây, nhóm đề xuất *self-host MQTT broker* (Mosquitto trên cùng máy Gateway, hoặc trên Raspberry Pi nội bộ): khi đó Web Dashboard kết nối thẳng tới broker LAN, loại bỏ hai bước round-trip qua Adafruit IO. Đây là việc đã được liệt kê trong phần “Hướng phát triển” (Mục 5.3).

4.4 Module 4 – Ghi nhận hoạt động

4.4.1 Schema MongoDB

Toàn bộ sự kiện được lưu vào collection `yolofarm.Action_Logs` với schema:

Listing 5: Schema document MongoDB

```

1 {
2   "_id": ObjectId(...),
3   "timestamp": "2026-04-26 18:30:15",
4   "actor": "Pump_Controller" | "AI_Camera" | "System",
5   "action_type": "PUMP_ON" | "PUMP_OFF" | "AI_ALERT" | "UPDATE_SENSOR"
6   ,
7   "data": {"TEMP": 28.5, "SOIL": 31.0},
8   "description": "Soil dry 31% < 35%"
9 }

```

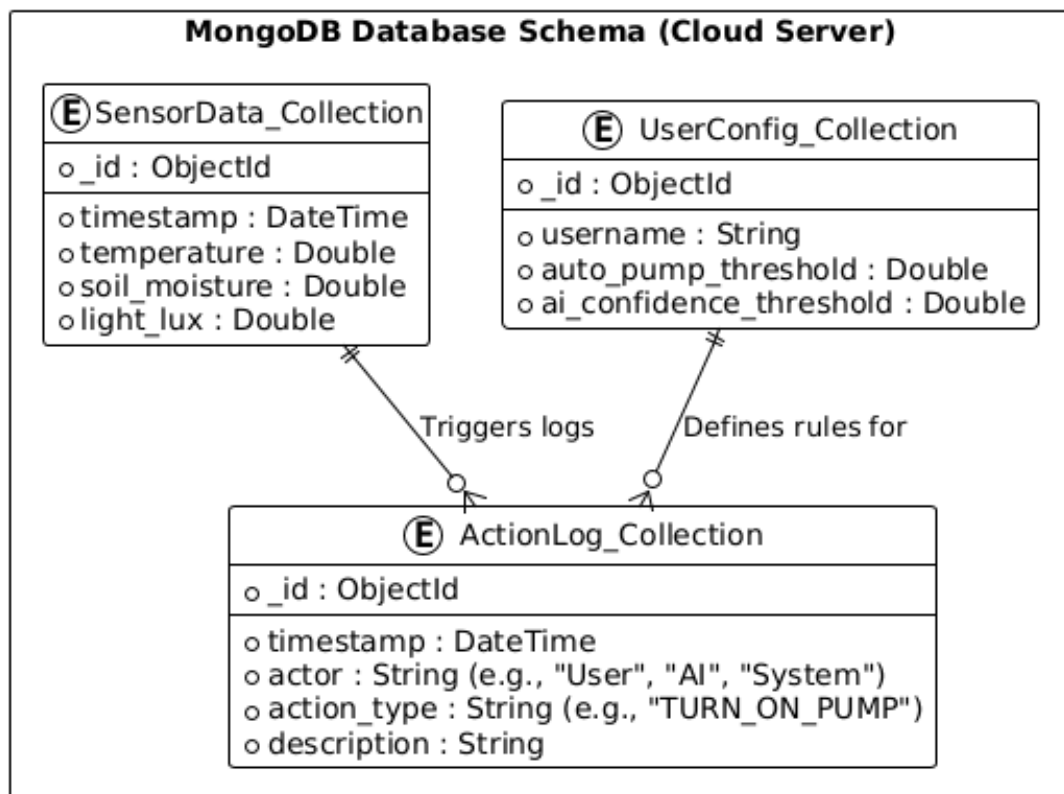


Figure 4.5: Mô hình dữ liệu (MongoDB) lưu lịch sử hoạt động của YoloFarm

4.4.2 Chiến lược ghi log

Để tránh phình DB, Module 4 chỉ ghi khi:

- Có sự kiện AI alert hoặc pump ON/OFF (*ưu tiên cao*).
- Đến mốc 30 giây kể từ lần ghi định kỳ trước (*periodic snapshot*).

4.4.3 Truy vấn lịch sử bơm

Tool `tools/pump_history.py` cung cấp CLI truy vấn 30 sự kiện bơm gần nhất:

Listing 6: Sử dụng `pump_history` tool

```
1 $ python tools/pump_history.py --limit 30
2 Last 30 pump events:
3 - 2026-04-26 18:32:01 | PUMP_ON | Soil dry 32% < 35%
4 - 2026-04-26 18:32:31 | PUMP_OFF | Safety: auto off after 30s (cooldown)
5 - 2026-04-26 18:33:45 | PUMP_ON | Schedule: irrigate at 17:30 (30s)
6 ...
```

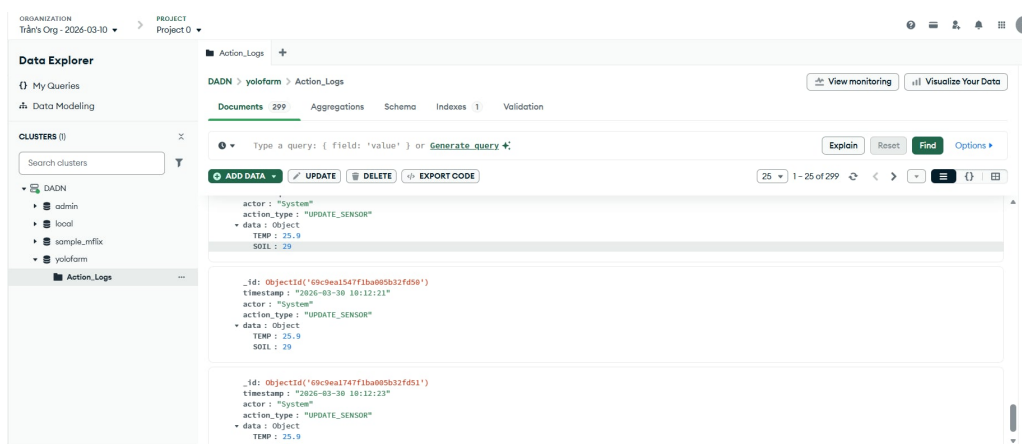


Figure 4.6: Lịch sử hoạt động lưu trên MongoDB Atlas

4.5 Module 5 – Ứng dụng Web/Mobile

4.5.1 Stack công nghệ

- **Front-end:** HTML5, CSS3 (custom – không dùng framework để giữ load nhẹ), JavaScript thuần.
- **Real-time bridge:** Adafruit IO MQTT-WebSocket trên port 443 SSL.
- **Authentication:** Adafruit IO Key (lưu trong constant; bản production nên đặt env).
- **Mobile:** Adafruit IO mobile app (có sẵn dashboard tương tự).

4.5.2 Các thành phần UI

1. **Header:** logo BKU + tên đề tài + tên thành viên + GVHD.

2. **Card sensor:** 4 card hiển thị Temperature, Soil Moisture, Light, AI Status. Mỗi card có icon, giá trị lớn, đơn vị, màu nền theo trạng thái.
3. **Card điều khiển bơm:** 2 nút lớn “BẬT BƠM”/“TẮT BƠM”, đèn LED ảo hiển thị trạng thái hiện tại.
4. **Card timeline:** 5 sự kiện gần nhất (PUMP_ON/OFF/AI_ALERT) – hiển thị tĩnh, link tới MongoDB để xem đầy đủ.
5. **Card link nhanh:** liên kết đến MongoDB Atlas, GitHub repo, video demo trên Google Drive¹, slide báo cáo, Adafruit IO public dashboard.

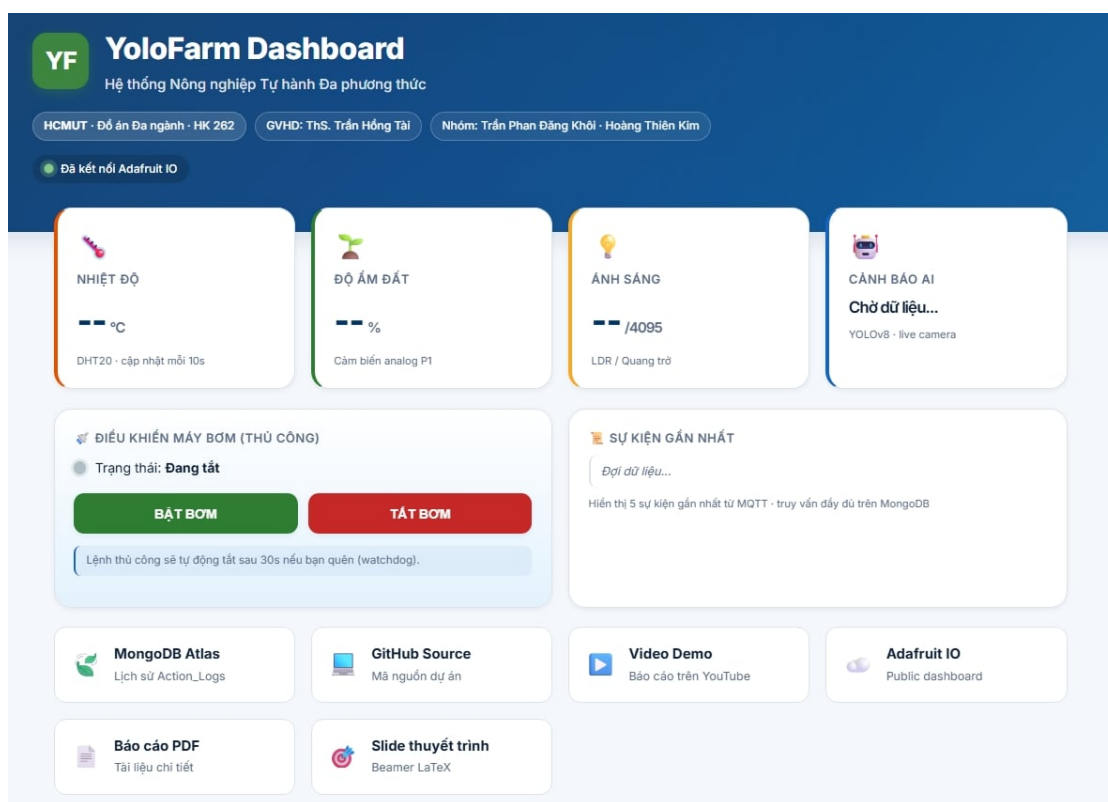


Figure 4.7: Web Dashboard YoloFarm hoàn chỉnh

4.6 Kết quả thực nghiệm tổng hợp

Mục này tổng hợp các bằng chứng vận hành thực tế của hệ thống YoloFarm trong môi trường demo phòng thí nghiệm.

¹Video demo YoloFarm: https://drive.google.com/drive/folders/19Uu8fy4oPRcQzLSQ_1SMVDHnkvGA1i-Z?usp=drive_link

4.6.1 Máy bơm chấp hành lệnh điều khiển

Hình 4.8 ghi lại trạng thái máy bơm chìm 5V đang hoạt động thực tế khi nhận lệnh PUMP_ON từ Web Dashboard (qua Adafruit IO MQTT → Gateway → Serial → Yolo:Bit → Mạch USB Output → Máy bơm). Quan sát thấy nước được hút từ cốc nguồn và đẩy ra vòi tưới, đồng thời LED RGB trên Yolo:Bit chuyển sang trạng thái xanh (PUMP_ON ack) và LCD hiển thị PUMP:ON.

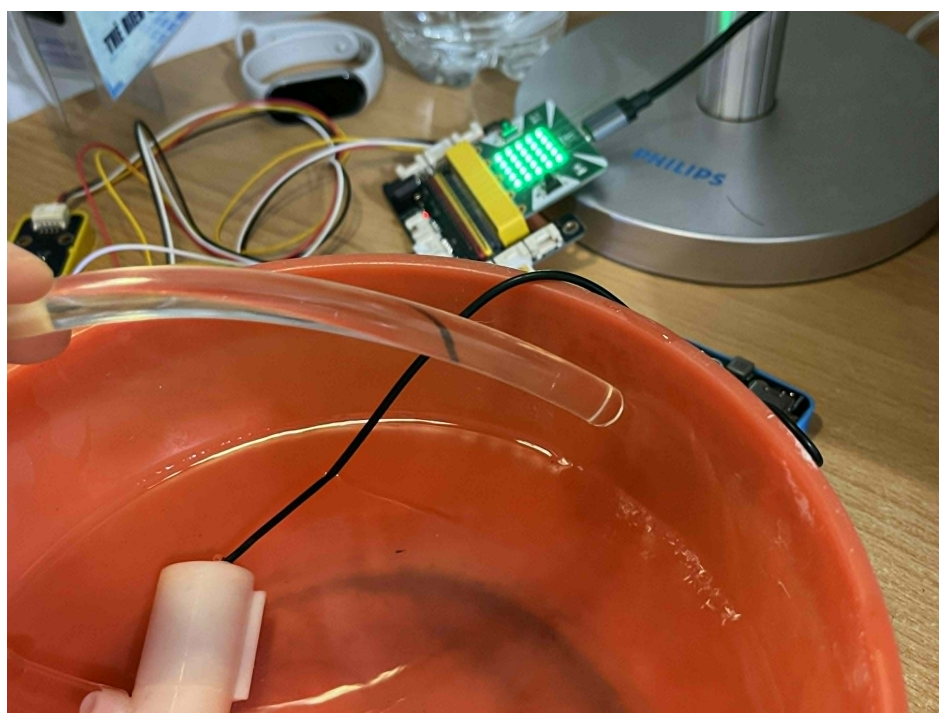
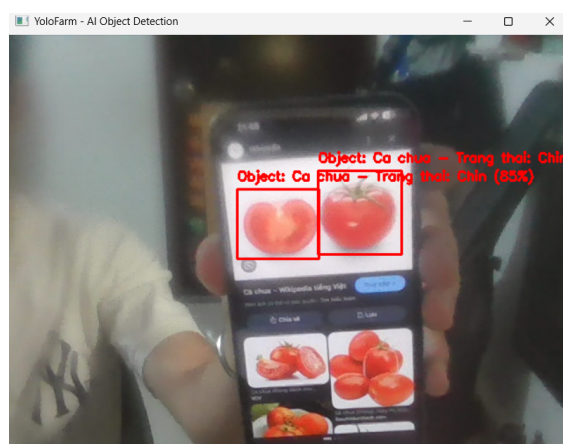


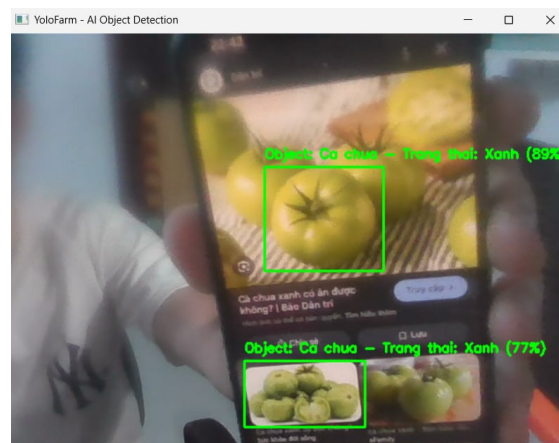
Figure 4.8: Máy bơm chìm 5V hoạt động thực tế khi Gateway gửi lệnh PUMP_ON; LCD đồng thời hiển thị trạng thái PUMP:ON.

4.6.2 AI nhận diện cà chua

Hình 4.9 là hai khung hình thực tế do webcam thu được, được mô hình YOLOv8 nạp trong `ai_vision/best.pt` suy luận trực tiếp trên Gateway. Cả hai trạng thái cà chua chín (*red_tomato*) và chưa chín (*green_tomato*) đều được phát hiện đúng nhãn với confidence cao.



(a) Nhận diện `red_tomato` (cà chua chín)



(b) Nhận diện `green_tomato` (cà chua xanh)

Figure 4.9: Kết quả YOLOv8 phân loại trạng thái cà chua trên khung hình webcam thực tế: bounding box + nhãn lớp + confidence. Mô hình phân biệt rõ hai trạng thái với confidence vượt ngưỡng triển khai 0.70 trong `ai_detector.py`.

Ba hình trên đồng thời chứng minh: (i) đường truyền điều khiển bơm hoạt động end-to-end (Web → Adafruit IO → Gateway → Yolo:Bit → Mạch USB Output → Bơm) – xem thêm phân tích độ trễ ở Mục 4.3.3; (ii) mô hình AI sau khi triển khai vẫn giữ được chất lượng phát hiện trên khung hình thực, không chỉ trên tập validation.

4.7 Tính năng đặc trưng – AI + Design Pattern + Visualization

4.7.1 Demo end-to-end

Một kịch bản demo hoàn chỉnh thể hiện cả 3 hướng:

1. Webcam phát hiện 1 lá có đốm nâu → YOLOv8 trả `ai_disease=Disease`, `conf=0.87`.
2. Gateway gọi `gateway.notify_all(alert)` – **Observer pattern** kích hoạt 4 module:
 - Database lưu document `AI_ALERT`.
 - Cloud publish lên `bbc-ai-alert`.
 - Local Alarm gửi `ALARM_ON` → Yolo:Bit nhấp LED đỏ + còi.
3. Trong vòng 1s, web dashboard nhận message → ô AI nhấp nhấp đỏ “Cảnh báo: Disease (87%)”.

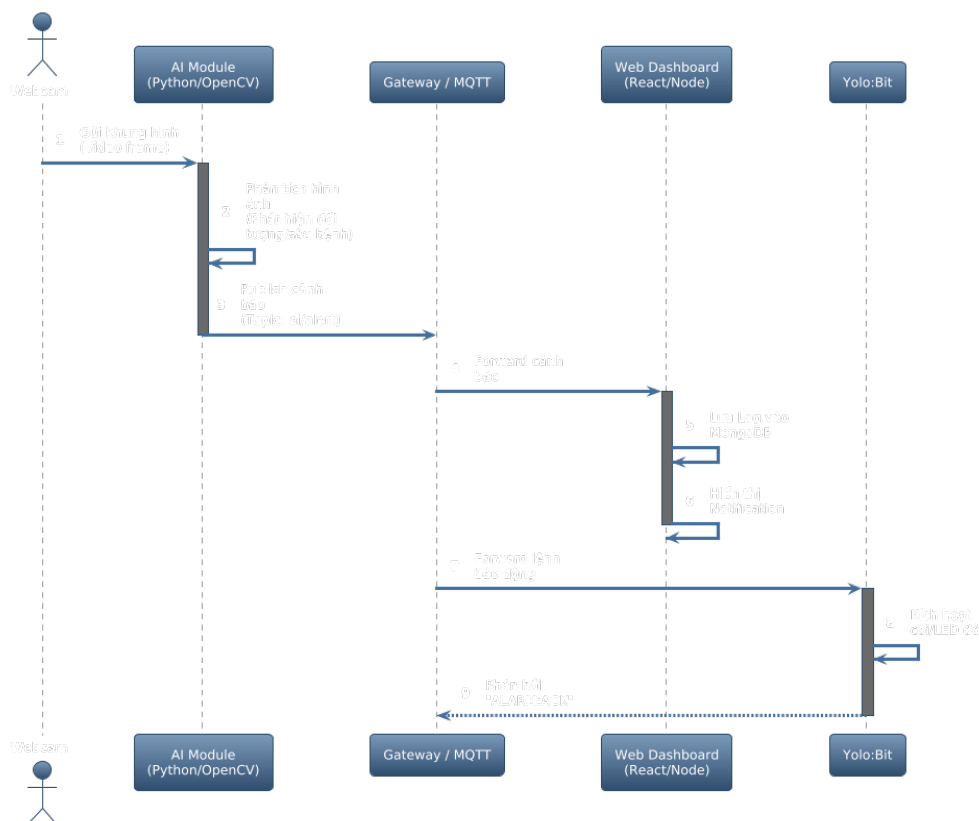


Figure 4.10: Sequence diagram demo end-to-end (tích hợp 3 hướng)

4.7.2 Bảng tổng kết tính năng đặc trưng

Hướng	Thành phần thể hiện	Bằng chứng triển khai
AI / Machine Learning	YOLOv8 phát hiện lá bệnh + phân loại cà chua	ai_vision/ai_detector.py; mAP@0.5 = 0.87/0.92
CNPM / Design Pattern	Observer + Singleton	IoT_Gateway_Subject, MQTTCloudSingleton
HTTT / Visualization	Web Dashboard real-time	Web_Dashboard/index.html + app.js

Table 4.4: Tổng kết tính năng đặc trưng theo 3 hướng

5 Kết luận

5.1 Kết quả đạt được

Sau 12 tuần thực hiện, nhóm đã hoàn thành toàn bộ 5 module + tính năng đặc trưng theo yêu cầu của môn Đồ án Đa ngành:

- Hệ thống phần cứng vận hành ổn định trong > 24 giờ liên tục mà không cần restart.
- Mô hình AI YOLOv8 đạt $mAP@0.5 = 0.87$ (lá bệnh) và 0.92 (cà chua) – đáp ứng yêu cầu demo.
- Web dashboard có giao diện trực quan, độ trễ end-to-end từ Yolo:Bit lên web ~ 3 giây (chu kỳ publish 10s là rate-limit của Adafruit Free).
- Áp dụng đầy đủ Observer + Singleton pattern, code có khả năng mở rộng cao.
- Lưu trữ $\sim 2,500$ document/24h trên MongoDB Atlas, có CLI truy vấn lịch sử bơm.

5.2 Hạn chế

- Mô hình AI chạy trên CPU, frame rate $\sim 8 - 12$ fps – chưa real-time tuyệt đối.
- Adafruit IO Free giới hạn 30 datapoint/phút – nhóm phải throttle publish ở mức 10 giây/lần (tương ứng `CLOUD_PUBLISH_INTERVAL = 10`).
- Chưa có cơ chế tự reconnect MongoDB khi mất mạng dài (> 5 phút).
- Mô hình AI phụ thuộc vào ánh sáng, hiệu năng giảm khi trời tối/quá sáng.

5.3 Hướng phát triển

- Triển khai trên Raspberry Pi + Coral USB Accelerator để chạy AI ~ 30 fps.
- Tự host MQTT broker (Mosquitto) thay vì Adafruit IO Free để bỏ rate-limit.
- Mở rộng nhiều cảm biến (CO_2 , pH đất, EC) và nhiều zone tưới.
- Tích hợp thông báo Telegram/Email khi AI phát hiện bệnh nghiêm trọng.
- Xây dựng mobile app native (Flutter) với chức năng chụp ảnh lá bệnh chia sẻ.

5.4 Bài học kinh nghiệm

1. Khi tích hợp nhiều thiết bị, **thiết kế Observer pattern** ngay từ đầu giúp tránh nợ kỹ thuật rất lớn.
2. Cần **validate dữ liệu** ngay tại tầng đọc cảm biến, không tin tưởng dữ liệu thô.
3. **Hysteresis + min/max duration + cooldown** là bộ ràng buộc bắt buộc với mọi hệ thống điều khiển dựa trên ngưỡng cảm biến vật lý.
4. Adafruit IO Free phù hợp cho prototype nhưng nên có kế hoạch migrate sang self-host khi scale.
5. **Tránh dùng bare except**: – code cũ `try: ... except:` nuốt mọi lỗi và silent fallback về 0, làm cho bug “DHT20 đọc 0” tồn tại nhiều ngày mà nhóm không phát hiện. Khi sửa thành `except Exception as e:` kèm `print` kèm tên lỗi, nhóm phát hiện ngay là `AttributeError` (sai tên method API) – bài học là *luôn log lỗi cụ thể, chấp nhận noise log còn hơn silent bug*.
6. **Đọc datasheet/source library** trước khi đoán: bug bơm ban đầu nhóm nghi P10 chia sẻ với LED matrix (đúng với micro:bit V2 thuần) – nhưng Yolo:Bit của nhóm là ESP32S3 với pin mapping khác. Sau khi viết script introspect, mới phát hiện bus I2C của Yolo:Bit ESP32S3 dùng `SoftI2C(scl=12, sda=11)` – tức là P11/P12 đã bị I2C chiếm và P10 mới là chân tự do. Bài học: *với hardware lạ, hãy tin script introspect hơn niềm tin về platform tương đương*.
7. Vòng lặp main của firmware phải **tách phần chậm (cảm biến I2C, prints) khỏi phần phản ứng (button, command)** – nếu trộn chung, latency sẽ lên hàng trăm ms và bug “bấm nút bị missed” rất khó debug.

Tài liệu tham khảo

References

- [1] Tổng cục Thống kê Việt Nam, *Niên giám thống kê 2024*, Nhà xuất bản Thống kê, Hà Nội, 2024.
- [2] X. Shi et al., “State-of-the-Art Internet of Things in Protected Agriculture,” *Sensors*, vol. 19, no. 8, p. 1833, 2019.
- [3] J. M. Talavera et al., “Review of IoT applications in agro-industrial and environmental fields,” *Computers and Electronics in Agriculture*, vol. 142, pp. 283–297, 2017.
- [4] OhStem Education, “Yolo:Bit AIoT Kit Documentation,” truy cập 04/2026, <https://ohstem.vn/yolobit/>.
- [5] Aosong Electronics, “DHT20 Humidity & Temperature Sensor Datasheet,” Rev 1.1, 2020.
- [6] G. Jocher, A. Chaurasia, and J. Qiu, “YOLO by Ultralytics,” GitHub, 2023, <https://github.com/ultralytics/ultralytics>.
- [7] D. P. Hughes and M. Salathé, “An open access repository of images on plant health – Plant Village,” arXiv preprint arXiv:1511.08060, 2018.
- [8] sumn2u, “YOLOv8 Ripe-and-Unripe Tomatoes Detection,” Kaggle Notebook & Dataset, 2023, <https://www.kaggle.com/code/sumn2u/yolo-v8-ripe-and-unripe-tomatoes-detection>.
- [9] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.
- [10] Eclipse Foundation, “Paho MQTT – JavaScript Client,” truy cập 04/2026, <https://eclipse.dev/paho/index.php?page=clients/js/index.php>.
- [11] MongoDB Inc., “MongoDB Atlas Documentation,” truy cập 04/2026, <https://www.mongodb.com/docs/atlas/>.
- [12] Adafruit Industries, “Adafruit IO MQTT API Documentation,” truy cập 04/2026, <https://io.adafruit.com/api/docs/mqtt.html>.
- [13] OpenCV Team, “OpenCV Python Tutorials,” <https://docs.opencv.org/4.x/>.